



Accélérateur de Talents Tech

COURS COMPLET

Formation Dev Web Full Stack



TypeScript

Du JavaScript typé à l'architecture robuste

Prérequis : **HTML** · **CSS** · **Git** · **JavaScript ES6+**

worketyamo.com · +237 674 197 190

Introduction

TypeScript est un sur-ensemble typé de JavaScript développé par Microsoft. Tout code JavaScript valide est du TypeScript valide — TypeScript y ajoute un système de types statiques, des interfaces, des enums et des fonctionnalités de POO avancées qui disparaissent à la compilation pour produire du JavaScript pur.

Ce module s'appuie directement sur le Module 06 JavaScript. Vous allez retrouver toutes les notions JS (variables, fonctions, classes, async/await, modules) et apprendre à les renforcer avec le typage statique de TypeScript.

Point	Détail
Langage	TypeScript 5.x (ES2023 target)
Prérequis	JavaScript ES6+ — Module 06 complet
Durée estimée	~15 heures (cours + exercices)
Environnement	VS Code + Node.js + ts-node
Projet fil rouge	TaskFlow API — version typée du projet JS

 Documentation officielle : tous les liens pointent vers la documentation TypeScript officielle et MDN.

[TypeScript — Documentation officielle](#)
[TypeScript Playground \(tester en ligne\)](#)
[TSConfig Reference](#)

CHAPITRE 01

Installation & Configuration

Mettre en place un environnement TypeScript professionnel

1.1 Installation

TypeScript s'installe via npm. En développement local, on l'installe comme dépendance de développement du projet.

```
# Installer TypeScript globalement (CLI tsc disponible partout)
npm install -g typescript

# Vérifier la version
tsc --version # Version 5.x

# Initialiser un projet TypeScript
mkdir mon-projet && cd mon-projet
npm init -y
npm install --save-dev typescript ts-node @types/node

# Générer le fichier de configuration
npx tsc --init
```

1.2 tsconfig.json — Configuration essentielle

Le fichier tsconfig.json contrôle le comportement du compilateur TypeScript. Voici une configuration recommandée pour un projet Node.js moderne.

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",           // Version JS produite
    "module": "CommonJS",        // Système de modules (Node.js)
    "lib": ["ES2022"],           // APIs disponibles
    "outDir": "./dist",          // Dossier de sortie
    "rootDir": "./src",          // Dossier source
    "strict": true,              // Active TOUS les checks stricts
    "esModuleInterop": true,     // Import style ES6 depuis CommonJS
    "skipLibCheck": true,        // Ignore les erreurs dans .d.ts
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,   // import data from "./data.json"
    "declaration": true,         // Génère les .d.ts
    "sourceMap": true            // Debugging en TypeScript
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

1.3 Compiler et exécuter

```
# Compiler tout le projet
npx tsc

# Compiler en watch mode (recompile à chaque sauvegarde)
npx tsc --watch

# Exécuter directement sans compiler (dev uniquement)
npx ts-node src/index.ts

# Scripts npm recommandés dans package.json
"scripts": {
  "build": "tsc",
  "dev": "ts-node src/index.ts",
  "start": "node dist/index.js"
}
```



L'option `strict: true` active `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes` et d'autres vérifications. Toujours l'activer sur un nouveau projet.

[TS — Options du compilateur](#)

CHAPITRE 02

Types de base & Annotations

Typer les variables, paramètres et valeurs de retour

2.1 Annotations de type

En TypeScript, on annote explicitement les types en ajoutant : type après le nom de la variable ou du paramètre. Le compilateur infère souvent le type — il n'est pas toujours nécessaire d'annoter.

```
// — Types primitifs —
const prenom: string = 'Alice'
const age: number = 25
const estActif: boolean = true
const rien: null = null
const nonDefini: undefined = undefined

// — Inférence — TypeScript déduit le type automatiquement —
const nom = 'Alice' // TypeScript sait que c'est string
const score = 100 // TypeScript sait que c'est number
// Pas besoin d'annoter quand la valeur initiale est claire

// — Quand annoter ? —
// 1. Paramètres de fonctions (toujours)
function saluer(prenom: string): string {
  return `Bonjour, ${prenom} !`
}

// 2. Variables déclarées sans valeur initiale
let resultat: string
resultat = 'OK'

// 3. Quand l'inférence serait trop large
let statut: 'TODO' | 'DONE' = 'TODO' // mieux que string
```

2.2 Les types TypeScript essentiels

Type	Description
string	Chaîne de caractères
number	Entier ET décimal (comme en JS)
boolean	true ou false
null	Absence intentionnelle de valeur
undefined	Valeur non assignée
any	Désactive le typage — à éviter absolument
unknown	Comme any mais sûr — doit être narrowed

never	Valeur qui n'existe jamais (fonction qui throw)
void	Fonction sans valeur de retour utile
object	N'importe quel objet non-primitif

```
// — any vs unknown —————
let x: any = 'texte'
x.methodeInventee() // Pas d'erreur - DANGEREUX

let y: unknown = 'texte'
y.methodeInventee() // Erreur TypeScript !
if (typeof y === 'string') {
  y.toUpperCase() // OK - TypeScript sait que c'est un string
}

// — void - Fonction sans retour —————
function logMessage(msg: string): void {
  console.log(msg) // Pas de return
}

// — never - Fonction qui ne termine jamais —————
function lancerErreur(message: string): never {
  throw new Error(message) // Jamais de return
}
```

2.3 Tableaux et Tuples

```
// — Tableaux —————
const nombres: number[] = [1, 2, 3, 4, 5]
const prenom: string[] = ['Alice', 'Bob']
const mixte: (string | number)[] = ['Alice', 25]

// Syntaxe alternative (moins courante)
const ids: Array<number> = [1, 2, 3]

// — Tuple - Tableau de longueur et types fixes —————
const coordonnee: [number, number] = [48.8566, 2.3522]

// Tuple nommé (plus lisible)
const entree: [id: number, nom: string, actif: boolean] = [1, 'Alice', true]

// Destructuration d'un tuple
const [latitude, longitude] = coordonnee

// Cas pratique : fonction retournant [data, error]
type Result<T> = [data: T | null, error: string | null]

function diviser(a: number, b: number): Result<number> {
  if (b === 0) return [null, 'Division par zéro']
  return [a / b, null]
}

const [resultat, erreur] = diviser(10, 2) // [5, null]
```


CHAPITRE 03

Interfaces & Types

Définir des contrats de données avec interface et type

3.1 Interfaces

Une interface définit la forme d'un objet — quelles propriétés il doit avoir et leur type. C'est le principal outil pour typer les données métier en TypeScript.

```
// — Interface de base —————
interface Tache {
  id: number
  titre: string
  statut: 'TODO' | 'IN_PROGRESS' | 'DONE' | 'CANCELLED'
  priorite: 'LOW' | 'MEDIUM' | 'HIGH' | 'URGENT'
  createdAt: Date
  completedAt: Date | null
  tags: string[]
}

// — Propriétés optionnelles (?) —————
interface CreerTacheDTO {
  titre: string
  priorite?: 'LOW' | 'MEDIUM' | 'HIGH' | 'URGENT' // Optionnel
  tags?: string[]
  dueDate?: Date
}

// — Propriétés en lecture seule (readonly) —————
interface Utilisateur {
  readonly id: string // Ne peut pas être modifié après création
  readonly createdAt: Date
  prenom: string
  email: string
  role: 'ADMIN' | 'USER' | 'VIEWER'
}

// — Utilisation —————
function afficherTache(tache: Tache): string {
  return `[${tache.priorite}] ${tache.titre} - ${tache.statut}`
}
```

3.2 Extension d'interfaces

```
// — extends — Hériter et étendre une interface —————
interface TacheBase {
  id: number
  titre: string
  createdAt: Date
}
```

```
interface TacheProjet extends TacheBase {
  projetId: string
  assignee: string
  commentaires: Commentaire[]
}

// Étendre plusieurs interfaces
interface TacheComplete extends TacheBase, TacheProjet {
  historique: HistoriqueItem[]
}

// — Interfaces pour fonctions —————
interface Compareur<T> {
  (a: T, b: T): number
}

const parPriorite: Compareur<Tache> = (a, b) =>
  PRIORITES.indexOf(b.priorite) - PRIORITES.indexOf(a.priorite)
```

3.3 Type aliases

Les type aliases permettent de nommer n'importe quel type — pas seulement des objets. Ils sont interchangeables avec les interfaces pour les objets, mais plus flexibles pour les types primitifs, unions et intersections.

```
// — Type alias simple —————
type Statut = 'TODO' | 'IN_PROGRESS' | 'DONE' | 'CANCELLED'
type Priorite = 'LOW' | 'MEDIUM' | 'HIGH' | 'URGENT'
type ID = string | number

// — Type alias objet (équivalent à interface) —————
type Point = {
  x: number
  y: number
}

// — Union types —————
type ResultatOK<T> = { success: true; data: T }
type ResultatErreur = { success: false; error: string; code: number }
type Resultat<T> = ResultatOK<T> | ResultatErreur

function creerTache(dto: CreerTacheDTO): Resultat<Tache> {
  if (!dto.titre.trim())
    return { success: false, error: 'Titre requis', code: 400 }
  return { success: true, data: { /* tache créée */ } as Tache }
}

// — Intersection types —————
type AvecTimestamps = { createdAt: Date; updatedAt: Date }
type AvecID = { id: string }
type EntiteBase = AvecID & AvecTimestamps

// — interface vs type — quand utiliser quoi ?
// interface : modèles de données, classes, APIs publiques
// type : unions, intersections, alias de primitifs, utilitaires
```

💡 Règle pratique : utilisez interface pour les objets représentant des entités (Utilisateur, Tache, Projet) et type pour les unions, les utilitaires et les types composites.

[TS — Interfaces](#)

[TS — Type aliases](#)

CHAPITRE 04

Fonctions Typées

Paramètres, valeurs de retour et surcharges

4.1 Typier les fonctions

```
// — Paramètres et valeur de retour —————
function additionner(a: number, b: number): number {
  return a + b
}

// — Arrow function typée —————
const multiplier = (a: number, b: number): number => a * b

// — Paramètre optionnel (?) —————
function saluer(prenom: string, civilite?: string): string {
  return civilite ? `${civilite} ${prenom}` : prenom
}

// — Valeur par défaut + inférence du type —————
function creerSlug(titre: string, separateur = '-'): string {
  return titre.toLowerCase().replace(/\s+/g, separateur)
}

// — Paramètre rest typé —————
function somme(...nombres: number[]): number {
  return nombres.reduce((acc, n) => acc + n, 0)
}

// — Type de fonction —————
type Compareur<T> = (a: T, b: T) => number
type Predicat<T> = (item: T) => boolean
type Transformateur<T, U> = (input: T) => U
```

4.2 Génériques (Generics)

Les génériques permettent d'écrire des fonctions et classes réutilisables qui fonctionnent avec n'importe quel type, tout en préservant les informations de type.

```
// — Generic de base —————
function premier<T>(tableau: T[]): T | undefined {
  return tableau[0]
}

const p1 = premier([1, 2, 3]) // type: number | undefined
const p2 = premier(['a', 'b']) // type: string | undefined

// — Contrainte générique (extends) —————
function afficherNom<T extends { nom: string }>(entite: T): string {
  return entite.nom
}
```

```
// — Plusieurs paramètres génériques —————
function transformer<T, U>(tableau: T[], fn: (item: T) => U): U[] {
  return tableau.map(fn)
}

const longueurs = transformer(['alice', 'bob'], s => s.length)
// longueurs: number[] - TypeScript infère le type de retour

// — Generic avec valeur par défaut —————
interface ApiReponse<T = unknown> {
  data: T
  status: number
  message: string
}

// — Fonctions utilitaires typées —————
function trouverParId<T extends { id: number | string }>(
  collection: T[],
  id: T['id']
): T | undefined {
  return collection.find(item => item.id === id)
}
```

4.3 Surcharges de fonctions

```
// — Overloads - plusieurs signatures —————
function formater(valeur: string): string
function formater(valeur: number): string
function formater(valeur: Date): string
function formater(valeur: string | number | Date): string {
  if (typeof valeur === 'string') return valeur.trim()
  if (typeof valeur === 'number') return valeur.toFixed(2)
  return new Intl.DateTimeFormat('fr-FR').format(valeur)
}

formater('  bonjour  ') // string
formater(3.14159)       // '3.14'
formater(new Date())    // '20/05/2026'
```

[TS — Fonctions](#)

[TS — Generics](#)

CHAPITRE 05

Classes TypeScript

Classes, modificateurs d'accès, interfaces et héritage

5.1 Classes avec modificateurs d'accès

TypeScript enrichit les classes ES6 avec des modificateurs d'accès (public, private, protected, readonly) et des propriétés de classe typées.

```
class Tache {
  // — Modificateurs d'accès —————
  public titre: string          // Accessible partout (défaut)
  public statut: Statut
  private readonly id: number // Privé + lecture seule
  private readonly createdAt: Date
  protected priorite: Priorite

  private static compteur = 0
  static readonly STATUTS: Statut[] = ['TODO', 'IN_PROGRESS', 'DONE']

  // — Shorthand constructor (déclare ET initialise) —————
  constructor(
    public titre: string,          // Syntaxe courte
    protected priorite: Priorite = 'MEDIUM',
  ) {
    Tache.compteur++
    this.id = Tache.compteur
    this.createdAt = new Date()
    this.statut = 'TODO'
  }

  // — Accesseurs —————
  get estTerminee(): boolean { return this.statut === 'DONE' }
  get idTache(): number { return this.id }

  set nouvellepriorite(val: Priorite) {
    const valides: Priorite[] = ['LOW', 'MEDIUM', 'HIGH', 'URGENT']
    if (!valides.includes(val)) throw new Error(`Priorité invalide`)
    this.priorite = val
  }

  // — Méthodes —————
  terminer(): this { // Retourne this pour le chaînage
    if (this.statut === 'DONE') throw new Error('Déjà terminée')
    this.statut = 'DONE'
    return this
  }

  toString(): string {
    return `[${this.id}] ${this.titre} - ${this.statut}`
  }
}
```

5.2 Implémenter des interfaces

```
// — Interface à implémenter —
interface Serialisable {
  toJSON(): Record<string, unknown>
  toString(): string
}

interface Validable {
  valider(): boolean
  readonly erreurs: string[]
}

// — Classe qui implémente plusieurs interfaces —
class TacheProjet extends Tache implements Serialisable, Validable {
  readonly erreurs: string[] = []

  constructor(
    titre: string,
    private readonly projetId: string,
    public assignee: string,
    priorite: Priorite = 'MEDIUM'
  ) {
    super(titre, priorite) // Obligatoire
    this.commentaires = []
  }

  private commentaires: Array<{ texte: string; auteur: string; date: Date }>

  ajouterCommentaire(texte: string, auteur: string): this {
    this.commentaires.push({ texte, auteur, date: new Date() })
    return this
  }

  valider(): boolean {
    if (!this.titre.trim()) this.erreurs.push('Titre requis')
    if (!this.projetId) this.erreurs.push('Projet requis')
    return this.erreurs.length === 0
  }

  toJSON(): Record<string, unknown> {
    return {
      id: this.idTache, titre: this.titre, statut: this.statut,
      projetId: this.projetId, assignee: this.assignee,
    }
  }
}
```

[TS — Classes](#)

CHAPITRE 06

Types Avancés

Union, intersection, utility types et type narrowing

6.1 Union et Intersection

```
// — Union type (|) — l'un OU l'autre —————
type IDType = string | number
type Statut = 'TODO' | 'IN_PROGRESS' | 'DONE' // Union littérale

function afficher(valeur: string | number): void {
  console.log(String(valeur))
}

// — Intersection type (&) — les deux à la fois —————
type AvecId = { id: string }
type AvecTimestamps = { createdAt: Date; updatedAt: Date }
type Entite = AvecId & AvecTimestamps

// — Discriminated Union (pattern essentiel) —————
type EvenementTache =
  | { type: 'CREEE'; tache: Tache }
  | { type: 'MISE_A_JOUR'; id: number; changements: Partial<Tache> }
  | { type: 'SUPPRIMEE'; id: number }

function traiterEvenement(evt: EvenementTache): void {
  switch (evt.type) {
    case 'CREEE': console.log('Nouvelle tâche:', evt.tache); break
    case 'MISE_A_JOUR': console.log('Update id:', evt.id); break
    case 'SUPPRIMEE': console.log('Supprimée id:', evt.id); break
  }
}
```

6.2 Utility Types — Les plus utiles

Utility Type	Description
Partial<T>	Toutes les propriétés de T deviennent optionnelles
Required<T>	Toutes les propriétés de T deviennent obligatoires
Readonly<T>	Toutes les propriétés de T sont en lecture seule
Pick<T, K>	Sélectionne un sous-ensemble de propriétés de T
Omit<T, K>	Exclut les propriétés K de T
Record<K, V>	Objet avec clés de type K et valeurs de type V
Exclude<T, U>	Exclut les membres de T qui sont assignables à U
Extract<T, U>	Garde les membres de T assignables à U

NonNullable<T>	Exclut null et undefined de T
ReturnType<F>	Extrait le type de retour d'une fonction
Parameters<F>	Extrait les types des paramètres d'une fonction
Awaited<T>	Extrait le type résolu d'une Promise

```
interface Tache {
  id: number; titre: string; statut: Statut; priorite: Priorite
}

// — Partial — DTO de mise à jour —————
type MettreAJourTacheDTO = Partial<Tache>
// Toutes les propriétés sont optionnelles → idéal pour PATCH

// — Pick — Sélectionner des champs —————
type TacheResume = Pick<Tache, 'id' | 'titre' | 'statut'>

// — Omit — Exclure des champs —————
type CreerTacheDTO = Omit<Tache, 'id' | 'createdAt'>

// — Record — Index map —————
type TachesParStatut = Record<Statut, Tache[]>
type Config = Record<string, string | number | boolean>

// — ReturnType — Inférer le type de retour —————
async function fetchTaches() { return [] as Tache[] }
type TachesPromise = ReturnType<typeof fetchTaches> // Promise<Tache[]>
type TachesData = Awaited<TachesPromise> // Tache[]
```

6.3 Type Narrowing

```
// — typeof narrowing —————
function traiter(valeur: string | number): string {
  if (typeof valeur === 'string') return valeur.trim() // string ici
  return valeur.toFixed(2) // number ici
}

// — instanceof narrowing —————
function gererErreur(err: unknown): string {
  if (err instanceof Error) return err.message
  if (typeof err === 'string') return err
  return 'Erreur inconnue'
}

// — Type guard (is) —————
function estTache(obj: unknown): obj is Tache {
  return typeof obj === 'object' && obj !== null && 'titre' in obj
}

const donnees: unknown = await fetch('/api/tache/1').then(r => r.json())
if (estTache(donnees)) {
  console.log(donnees.titre) // TypeScript sait que c'est une Tache
}
```

```
// — Assertion de type (as) — utiliser avec prudence —————  
const input = document.querySelector('#titre') as HTMLInputElement  
// Seulement quand vous êtes certain du type — sinon utiliser un type guard
```

[TS — Utility Types](#)[TS — Narrowing](#)

CHAPITRE 07

Enums & Littéraux

Énumérations et types littéraux pour les valeurs constantes

7.1 Enums

Les enums permettent de définir un ensemble de constantes nommées. TypeScript offre des enums numériques, string et const.

```
// — Enum string (recommandé) —————
enum Statut {
  TODO      = 'TODO',
  IN_PROGRESS = 'IN_PROGRESS',
  DONE      = 'DONE',
  CANCELLED  = 'CANCELLED',
}

enum Priorite {
  LOW      = 'LOW',
  MEDIUM  = 'MEDIUM',
  HIGH     = 'HIGH',
  URGENT   = 'URGENT',
}

// — Utilisation —————
const tache = { statut: Statut.TODO, priorite: Priorite.HIGH }
if (tache.statut === Statut.DONE) { /* ... */ }

// — Const enum (effacé à la compilation – plus performant) —
const enum Direction { NORD = 'N', SUD = 'S', EST = 'E', OUEST = 'O' }

// — Alternative moderne : union littérale (souvent préférable) —
type StatutType = 'TODO' | 'IN_PROGRESS' | 'DONE' | 'CANCELLED'
// Avantage : pas de dépendance runtime, plus simple

// — Object as const (pattern courant) —————
const STATUTS = {
  TODO:      'TODO',
  IN_PROGRESS: 'IN_PROGRESS',
  DONE:      'DONE',
} as const

type StatutValues = typeof STATUTS[keyof typeof STATUTS]
// 'TODO' | 'IN_PROGRESS' | 'DONE'
```

7.2 Enums numériques et itération

```
// — Enum numérique (auto-incrémenté) —————
enum CodeHTTP {
  OK          = 200,
  CREATED     = 201,
```

```
BAD_REQUEST      = 400,
UNAUTHORIZED     = 401,
NOT_FOUND        = 404,
INTERNAL_SERVER_ERROR = 500,
}

function repondre(code: CodeHTTP, message: string) {
  return { code, message }
}

repondre(CodeHTTP.OK, 'Tâche créée') // { code: 200, message: '...' }

// — Itérer sur un enum string —————
Object.values(Statut).forEach(s => console.log(s))
// 'TODO', 'IN_PROGRESS', 'DONE', 'CANCELLED'
```

💡 Préférez les union types ('TODO' | 'IN_PROGRESS') aux enums string pour les nouvelles bases de code — ils produisent du code plus léger et s'intègrent mieux avec JSON.

[TS — Enums](#)

CHAPITRE 08

Async TypeScript

Promises, async/await et fetch avec les types

8.1 Typer les Promises et async/await

Les concepts async/await du Module 06 JavaScript restent identiques. TypeScript y ajoute le typage du résultat des Promises et des erreurs.

```
// — Promise<T> — typer la valeur résolue —————
function attendre(ms: number): Promise<void> {
  return new Promise(resolve => setTimeout(resolve, ms))
}

async function getTache(id: number): Promise<Tache> {
  const res = await fetch(`/api/taches/${id}`)
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  return res.json() as Promise<Tache> // assertion nécessaire avec fetch
}

// — Typer les erreurs dans catch —————
async function charger(): Promise<Tache[]> {
  try {
    return await getTaches()
  } catch (err: unknown) { // err est 'unknown' avec strict
    if (err instanceof Error) {
      console.error(err.message)
    }
    return []
  }
}
```

8.2 Fetch API typée

```
// — Interface pour les réponses API —————
interface ApiReponse<T> {
  data: T
  message: string
  status: number
}

interface ErreurApi {
  message: string
  code: string
  details?: Record<string, string>
}

// — Client HTTP générique et typé —————
async function api<T>(url: string, options: RequestInit = {}): Promise<T> {
  const token = localStorage.getItem('token')
  const res = await fetch(url, {
```

```
...options,
headers: {
  'Content-Type': 'application/json',
  ...(token && { Authorization: `Bearer ${token}` }),
  ...options.headers,
},
body: options.body
  ? JSON.stringify(options.body as Record<string, unknown>)
  : undefined,
})

if (!res.ok) {
  const err: ErreurApi = await res.json().catch(() => ({
    message: 'Erreur réseau', code: 'NETWORK_ERROR',
  }))
  throw Object.assign(new Error(err.message), { status: res.status, code: err.code
})
}

return res.status === 204 ? (null as T) : (res.json() as Promise<T>)
}

// — Utilisation typée —————
const tache = await api<Tache>('/api/taches/1')
const nouvelleTache = await api<Tache>('/api/taches', {
  method: 'POST',
  body: { titre: 'Nouvelle tâche', priorite: 'HIGH' } as any,
})
```

[TS — Async / Await](#)[MDN — Fetch API](#)

CHAPITRE 09

Modules TypeScript

Organisation du code avec import/export typés

9.1 Export et Import

Les modules ES6 du Module 06 fonctionnent de façon identique en TypeScript. Le compilateur vérifie les types à travers les imports.

```
// — types.ts — Centraliser les types —————
export type Statut = 'TODO' | 'IN_PROGRESS' | 'DONE' | 'CANCELLED'
export type Priorite = 'LOW' | 'MEDIUM' | 'HIGH' | 'URGENT'

export interface Tache {
  id: number
  titre: string
  statut: Statut
  priorite: Priorite
  createdAt: Date
}

export type CreerTacheDTO = Omit<Tache, 'id' | 'createdAt'>
export type MettreAJourTacheDTO = Partial<CreerTacheDTO>

// — taches.service.ts —————
import type { Tache, CreerTacheDTO } from './types'
// 'import type' — importe uniquement pour le typage (effacé à la compile)

export async function getTaches(): Promise<Tache[]> {
  return api<Tache[]>('/api/taches')
}

export async function creerTache(dto: CreerTacheDTO): Promise<Tache> {
  return api<Tache>('/api/taches', { method: 'POST', body: dto as any })
}

// — app.ts —————
import type { Tache } from './types'
import { getTaches, creerTache } from './taches.service'

const taches: Tache[] = await getTaches()
```

9.2 Structure recommandée d'un projet TypeScript

```
src/
├─ index.ts           ← Point d'entrée
├─ types/
│   ├─ index.ts       ← Re-export central
│   ├─ tache.types.ts ← Interfaces et types Tache
│   └─ api.types.ts   ← Types de réponses API
└─ services/
```

```
|   ├── api.client.ts      ← Wrapper fetch générique  
|   ├── tache.service.ts  ← Appels API tâches  
|   └── auth.service.ts   ← Authentification  
|── utils/  
|   ├── date.utils.ts     ← Formatage de dates  
|   └── validation.ts     ← Fonctions de validation  
└── constants/  
    └── index.ts          ← Constantes de l'app
```

💡 Utilisez 'import type' pour les imports purement typés. Cela améliore les performances de compilation et évite les dépendances circulaires.

[TS — Modules](#)

CHAPITRE 10

Projet Final & Cheat Sheet

Récapitulatif et projet intégrateur TypeScript

10.1 Projet — TaskFlow API Typée

💡 **Projet :** Reprendre le projet TaskFlow UI du Module 06 et le convertir intégralement en TypeScript.
Durée estimée : 3 à 5 heures.

Fonctionnalités à typer

- Toutes les interfaces de données : Tache, Utilisateur, Projet, Commentaire
- Le client HTTP générique `api<T>()` avec gestion d'erreurs typée
- Les fonctions de manipulation de tableaux (`.map`, `.filter`, `.reduce`) avec génériques
- Les classes `Tache` et `TacheProjet` avec modificateurs d'accès
- Un enum ou union type pour `Statut` et `Priorite`
- Les fonctions de validation retournant `Resultat<T>` (union discriminée)
- Le store d'état global typé avec `Record<Statut, Tache[]>`

Contraintes techniques

- `strict: true` dans `tsconfig.json` — aucun `any` non justifié
- Tous les paramètres de fonctions doivent être annotés
- Utiliser des Utility Types (`Partial`, `Pick`, `Omit`) pour les DTOs
- Au moins une fonction générique avec contrainte (`extends`)
- Discriminated union pour les événements ou les erreurs

10.2 Cheat Sheet TypeScript

Types de base

```
const x: string = 'texte'
const y: number = 42
const z: boolean = true
const t: string[] = ['a', 'b']
const u: [string, number] = ['alice', 25] // tuple
function f(a: string): void {}
const fn: (a: number) => string = n => String(n)
```

Interface & type

```
interface Entite { id: string; nom: string }
interface Entite extends Base { extra: number }
type Union = 'A' | 'B' | 'C'
```

```
type Intersection = TypeA & TypeB
```

Utility types

```
Partial<T>           // Toutes les props optionnelles
Required<T>         // Toutes les props obligatoires
Readonly<T>         // Toutes les props readonly
Pick<T, 'a' | 'b'>   // Garder seulement a et b
Omit<T, 'a' | 'b'>   // Tout sauf a et b
Record<K, V>        // Objet { [K]: V }
ReturnType<typeof f> // Type de retour de la fonction f
Awaited<Promise<T>> // T
```

Generics

```
function id<T>(x: T): T { return x }
function first<T extends any[]>(arr: T): T[0] { return arr[0] }
interface Box<T = string> { value: T }
```

Narrowing

```
if (typeof x === 'string') { /* x: string */ }
if (x instanceof Error) { /* x: Error */ }
if ('prop' in obj) { /* obj has prop */ }
function isUser(x: unknown): x is User { return typeof x === 'object' && x !== null && 'id' in x }
```

10.3 Documentation officielle

[TypeScript Handbook \(référence complète\)](#)

[TypeScript Playground](#)

[TSConfig Reference](#)

[TS — Types de base](#)

[TS — Generics](#)

[TS — Utility Types](#)

[TS — Classes](#)

[TS — Modules](#)

[TS — Narrowing](#)

[TS — Enums](#)



TypeScript maîtrisé !

Vous avez couvert tout TypeScript moderne : types, interfaces, génériques, classes, utility types, async/await et modules. Prochaine étape : Node.js + Express !

◇ Worketyamo · worketyamo.com · +237 674 197 190